

Ocelot3

Kishore Senthil Kumar

Aug 22, 2025



Agenda

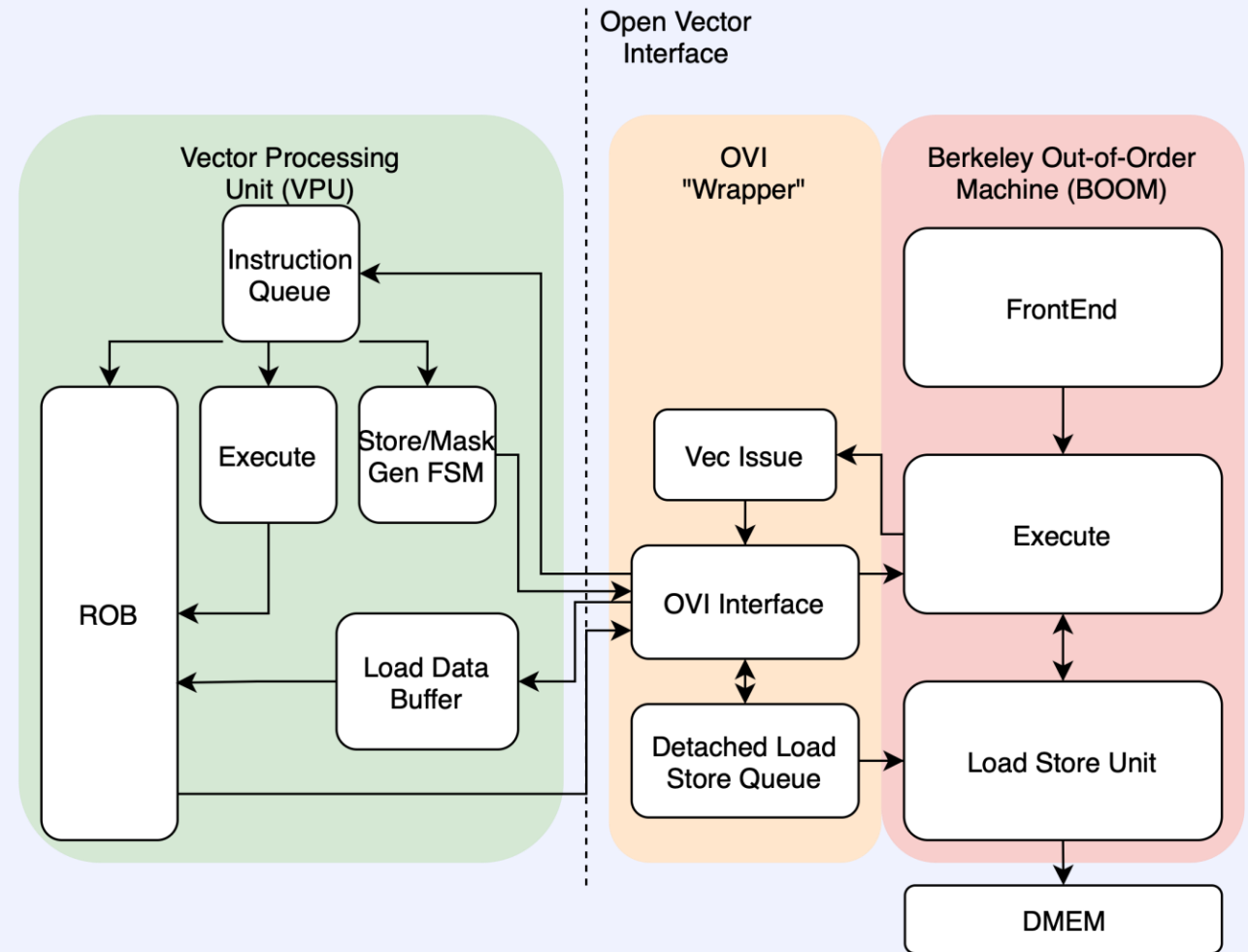
- Ocelot 2: Architectural Overview and Limitations
- Ocelot 3: Goals and Design Objectives
- Speculative Execution Enhancement
- Microarchitectural Restructuring of OVI Load/Store Generators
- Segment Load/Store: Comprehensive Support and Implementation
- Future Roadmap: Precise Trap Support
- AI tools, Misc.

Ocelot 2: Architectural Overview and Limitations



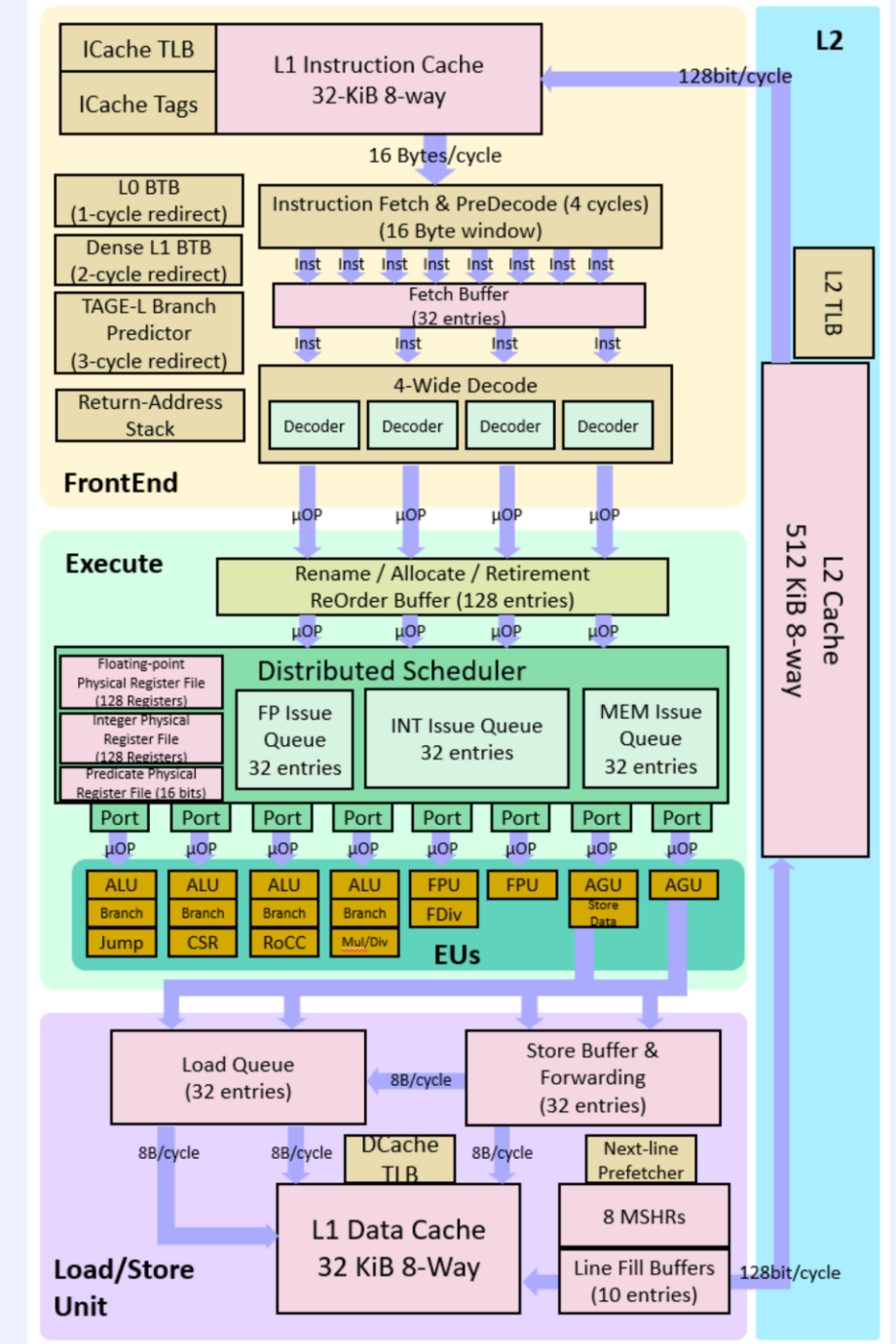
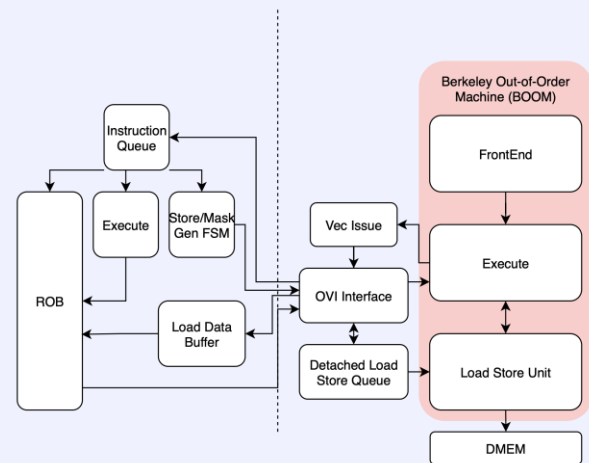
The Ocelot Project

- Open-Source Project:
<https://github.com/tenstorrent/riscv-ocelot/tree/bobcat>
- A vector extension to the Boom core
- VPU inspired by Baby RISC-V
- VPU connected through OVI protocol
- VPU shares memory interface with CPU



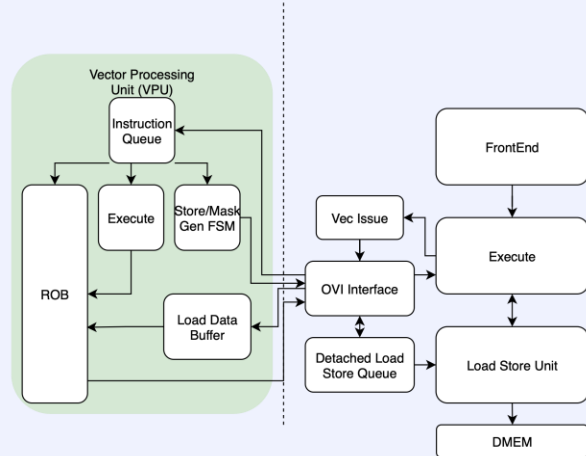
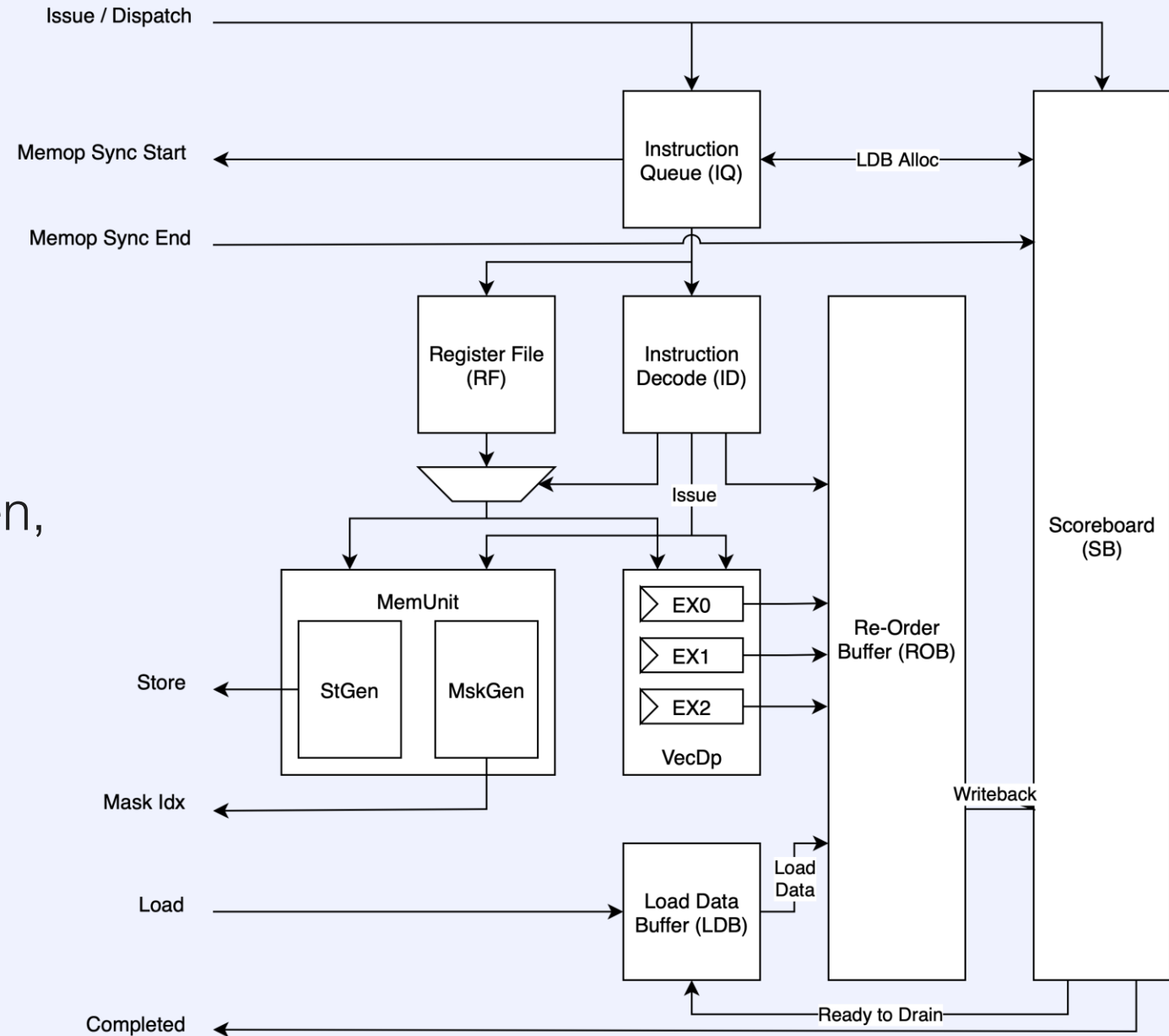
Berkeley Out of Order Machine (BOOMv3)

- Developed at UC Berkeley's Computer Architecture Lab
- Open-source processor design in Chisel hardware language
- Part of the Chipyard integrated design framework
- Ocelot Enhancements:
 - Extended ROB side of execute
 - Extended Load/Store Queue (LSQ) in the LSU



The Vector Unit

- Extracted from Baby RISC-V architecture (lightweight)
- No independent memory interface
- Not truly out-of-order: no RS
- Features Load Data Buffer, Store Gen, and Mask Gen
- Additional "scoreboard" tracks metadata for in-flight instructions



Open Vector Interface (OVI)

- Open-source Communication protocol by SemiDynamics for CPU-VPU interaction
- Decouples VPU from core: simple VPU swap
- Supports speculation (DISPATCH)
- Separate memory operation buses (LOAD/STORE)
- Supports Exception and Precise Trap
- Extended to support segment LS

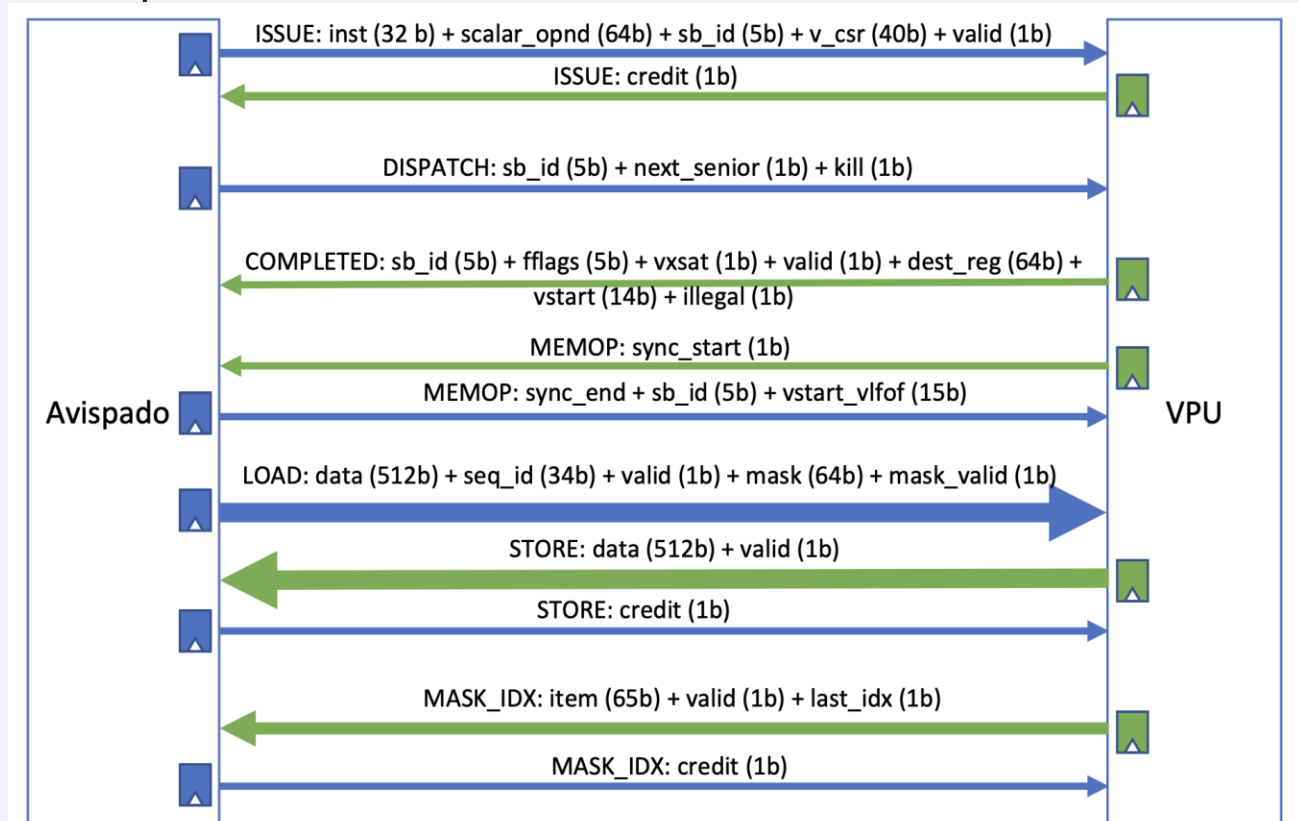
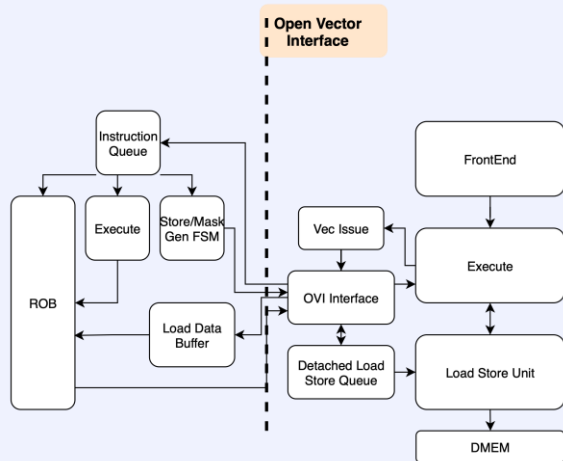


Figure 1. Signals and buses connecting Avispado to the VPU

Memory Buses through the OVI

- Supports strided and indexed memory operations
- Handles masked and unmasked vector loads/stores
- Uses sequence ID for out-of-order data transmission (data level and instruction level)
- Implements credit-based flow control

```
- seq_id[4:0] = v_reg
- seq_id[15:5] = el_id
- seq_id[21:16] = el_off
- seq_id[28:22] = el_count
- seq_id[33:29] = sb_id
```

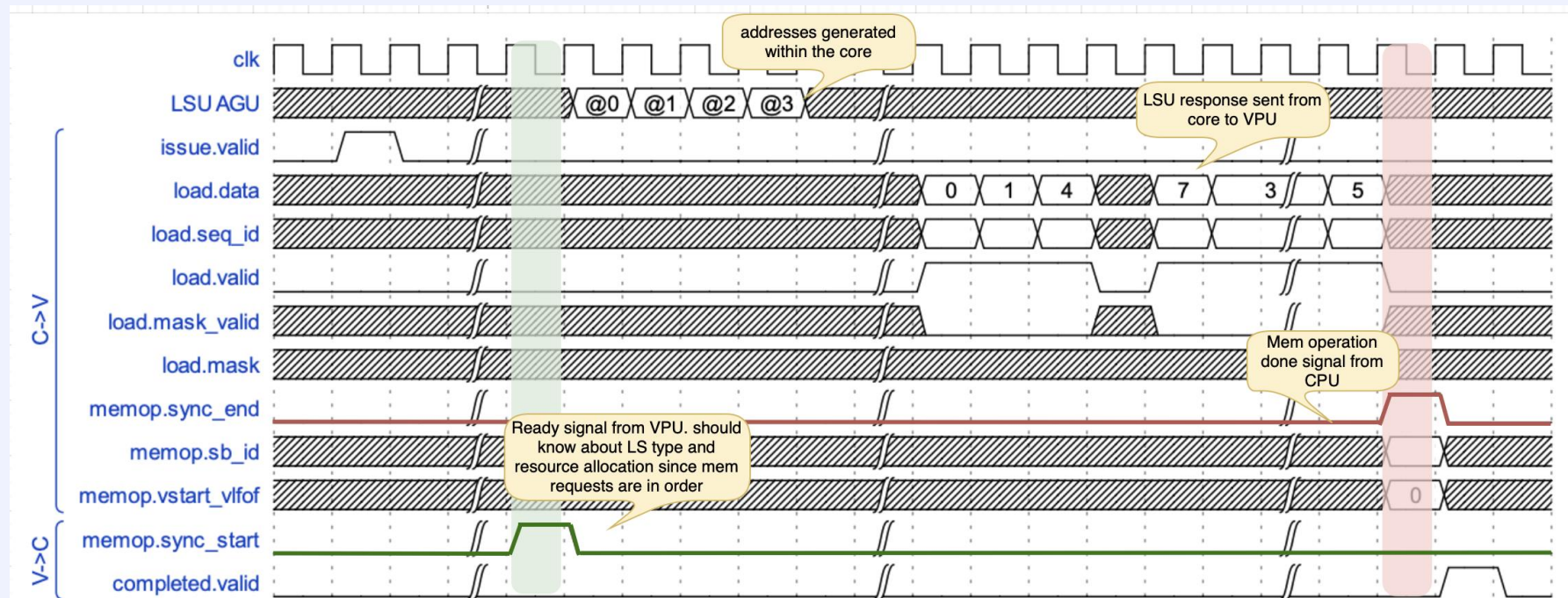


Figure 9. Strided load instruction



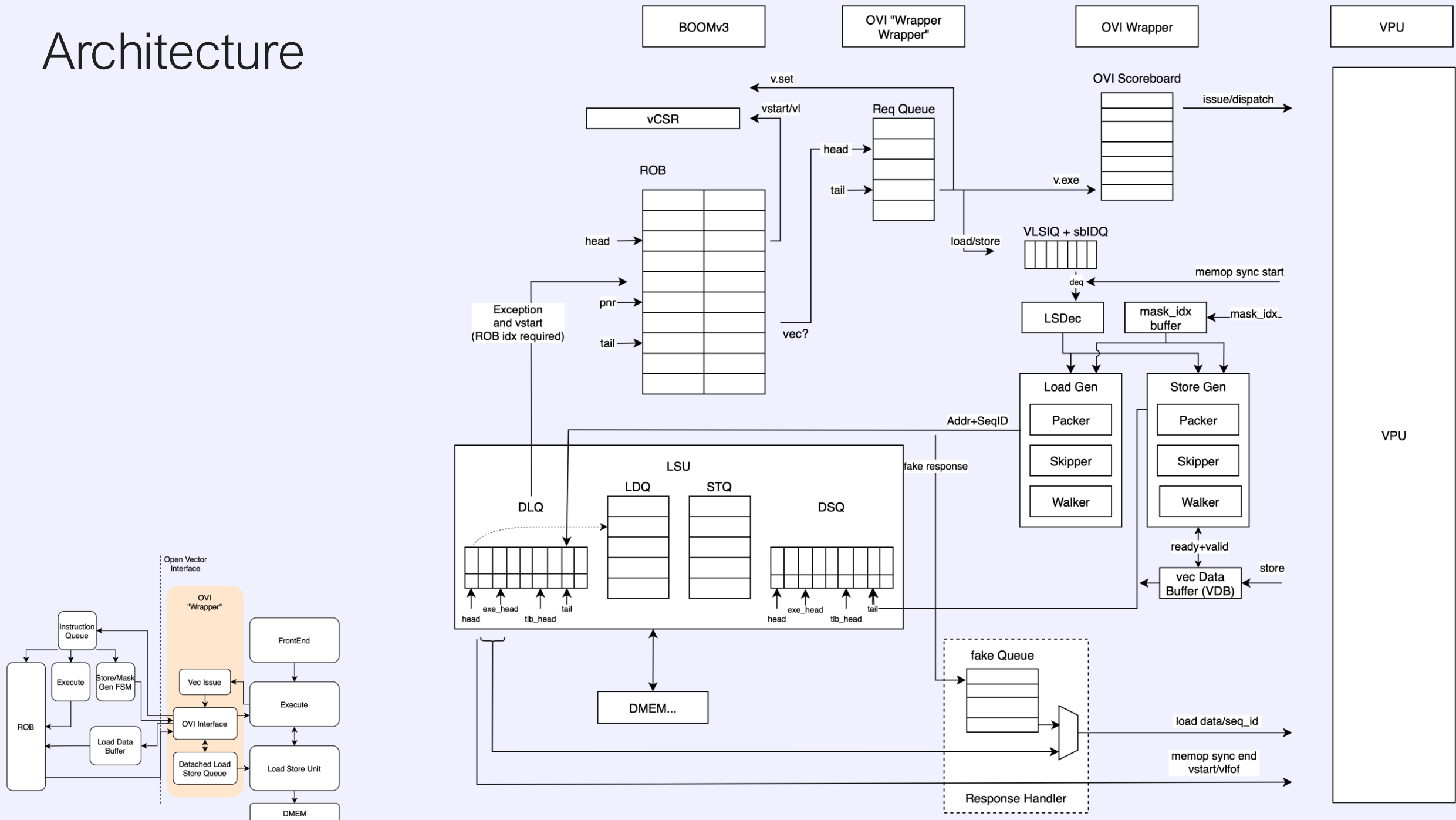
Ocelot 3: Goals and Design Objectives



Objectives

- Main Goal: Full RVV “V” 1.0 compliance
- Support for Segmented Loads / Stores
- Support for Precise Trap
- Complete missing arithmetic units: vdiv, vfddiv
- Keep OVI compliant as much as possible
- Enable Speculation

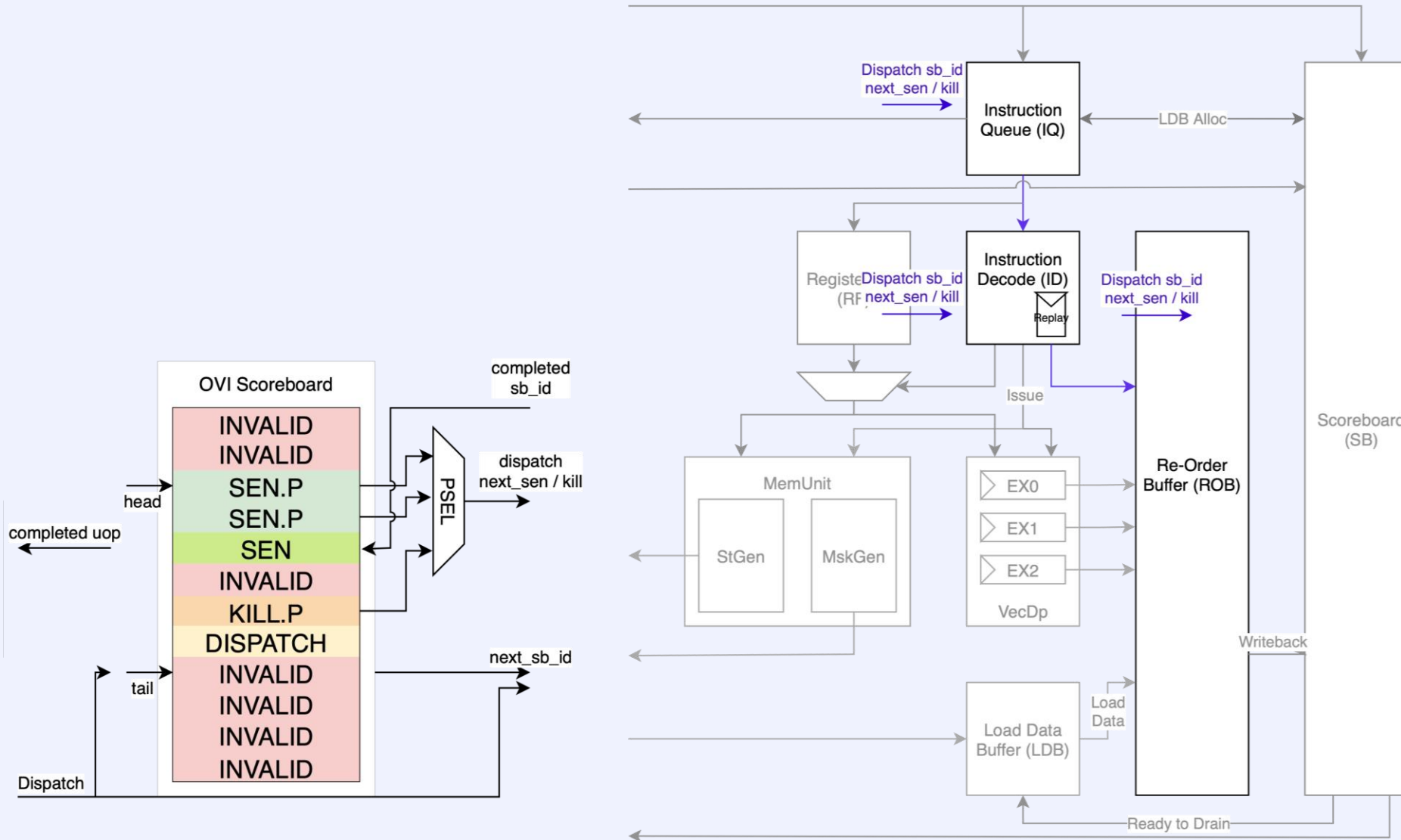
Architecture



Enabling Non-Load/Store Speculative Execution

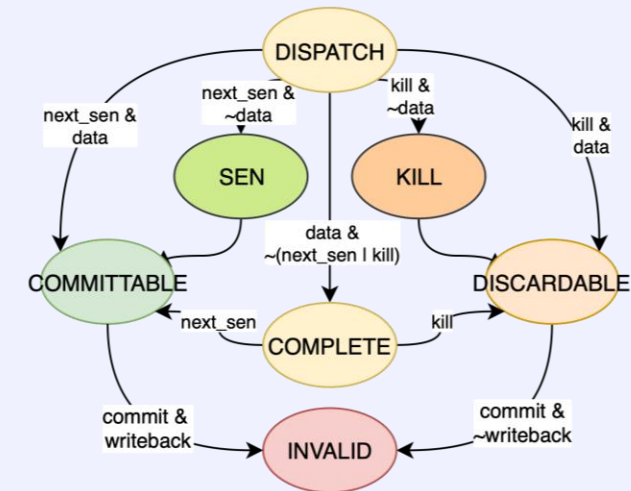


Enabling speculation



The FSM

After instructions enqueue into the ROB, they might have the final result and/or might have the dispatch decision. this is recorded by their state in

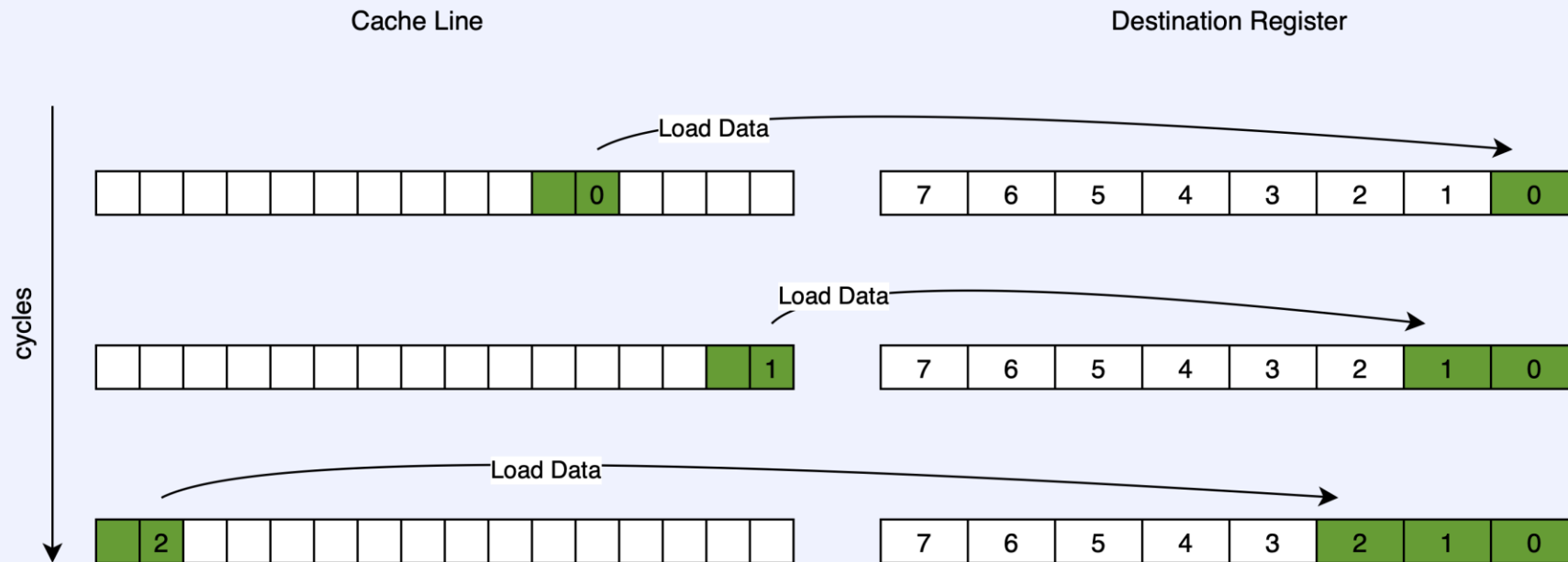


Restructuring of OVI Load/Store Generators



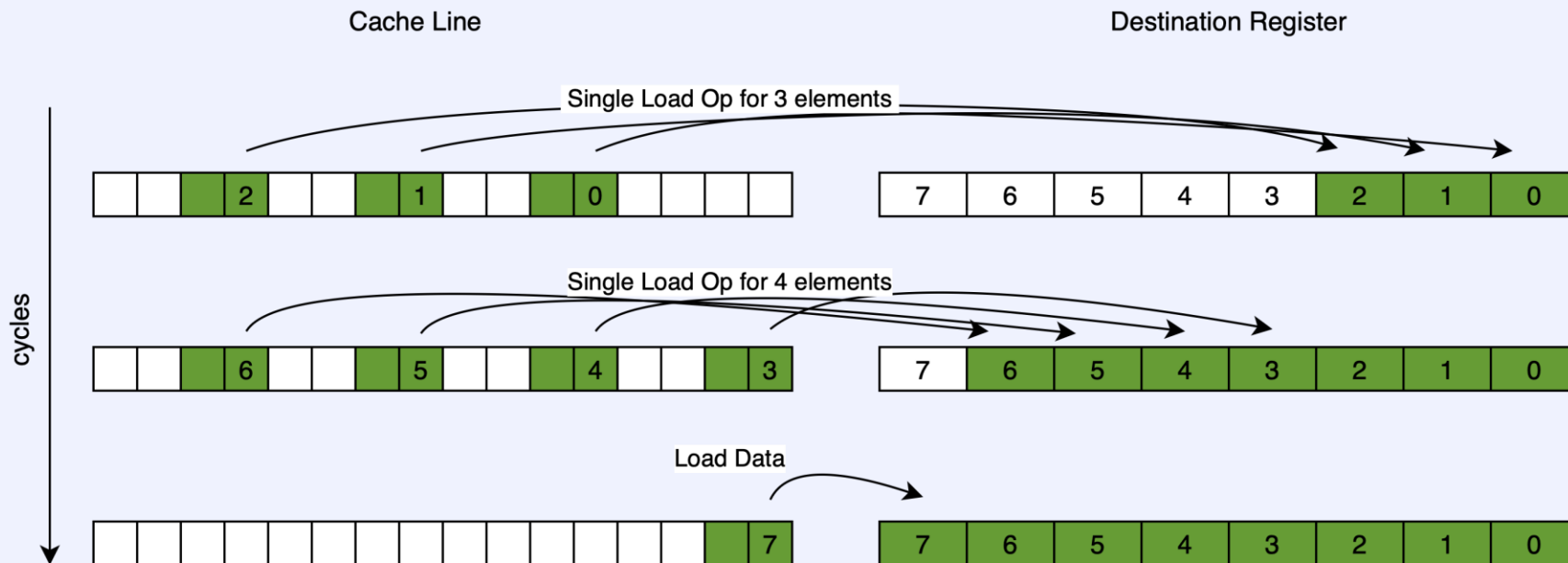
Walking through elements

- Processes elements sequentially (naive approach)
- Done for indexed loads or loads with bad strides



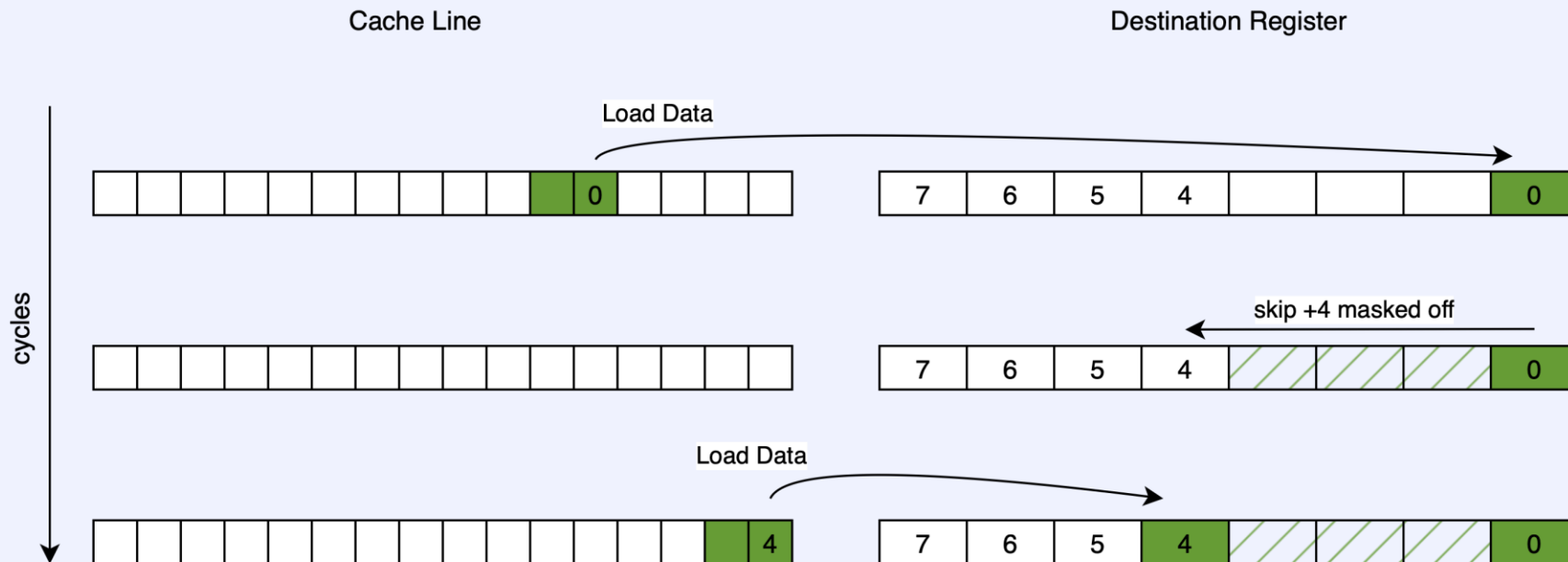
Packing elements

- Marches forward to see the maximum number of elements whose data fits within the cache line of the current address.
- Used for Good Strides (+1EEW, +2EEW, +4EEW)



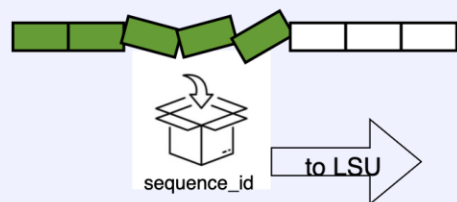
Skipping past elements

- Looks past contiguous 0s in mask buffer to skip unwanted elements. Processes skips in powers of 2 for efficiency.



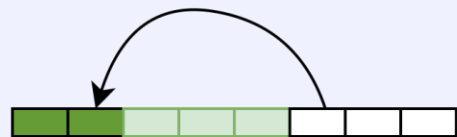
Packing

- Checks for the following:
- v_reg boundary
 - vl_boundary
 - dmem boundary/alignment
 - mask_idx buffer length
 - elem_id (segment only)



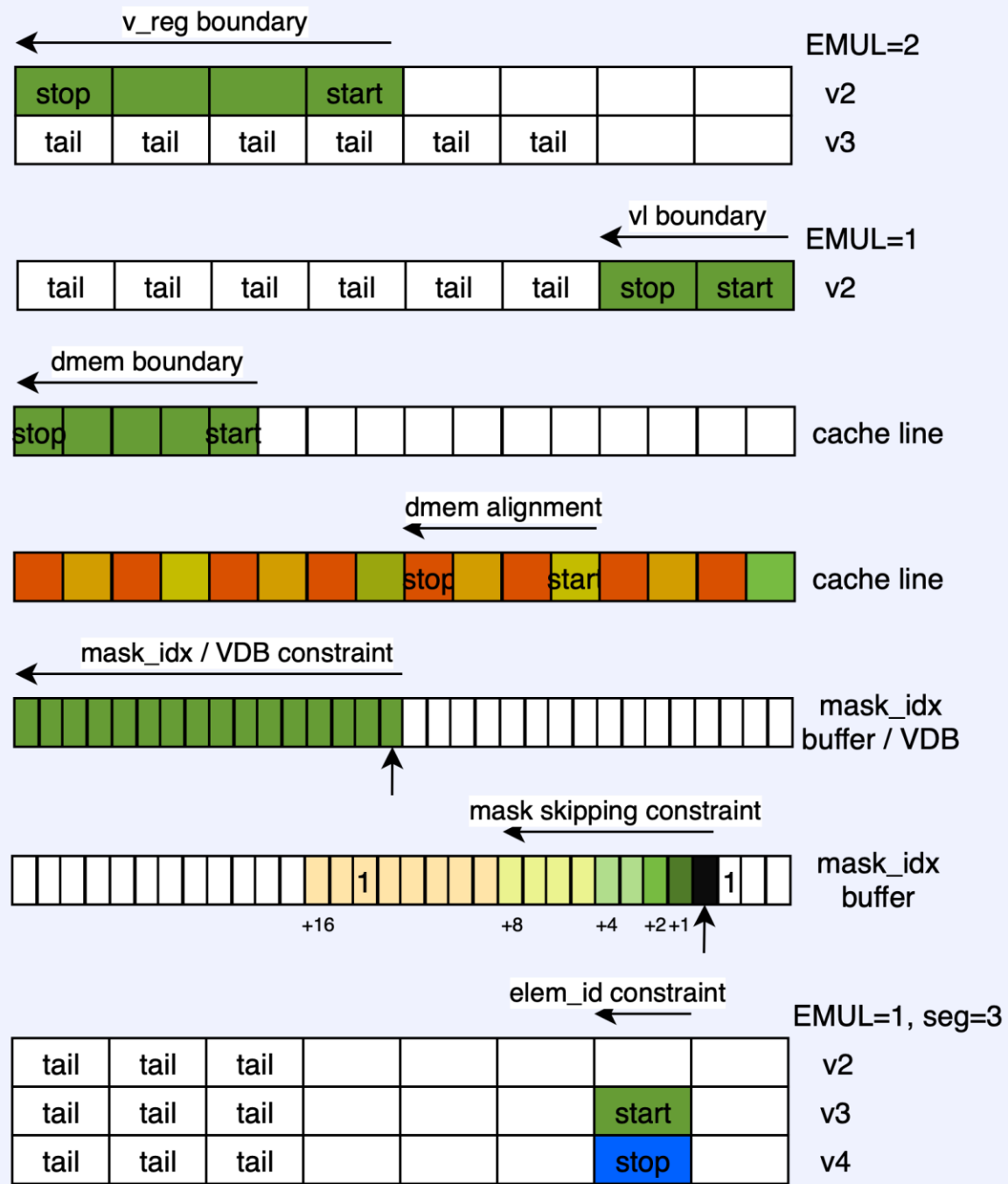
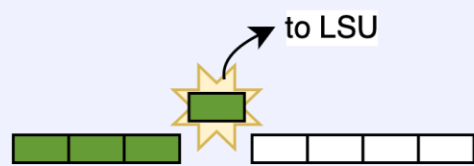
Skipping

- Checks for the following:
- mask_idx buffer length
 - mask skipping constraint
 - elem_id (segment only)
 - dmem (segment only)



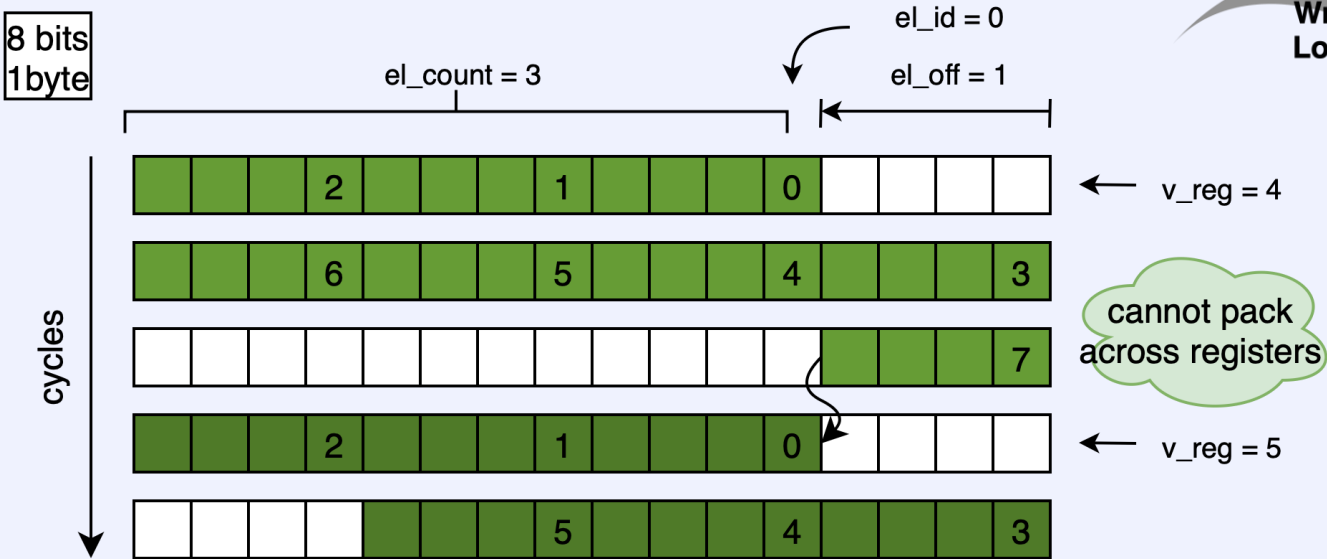
Walking

- Checks for the following:
- elem_id (segment only)
 - dmem (segment only)



Unit Stride Packing Example

VPU will know how many bytes an element is since that info is decoded



Larg/Mega Bobcat DMEM_W= 128bits

```
li x9, 14
vsetvli x8, x9, e32, m2, tu, mu
li x6, 0x80002014
vle32.v v4, (x6)
```

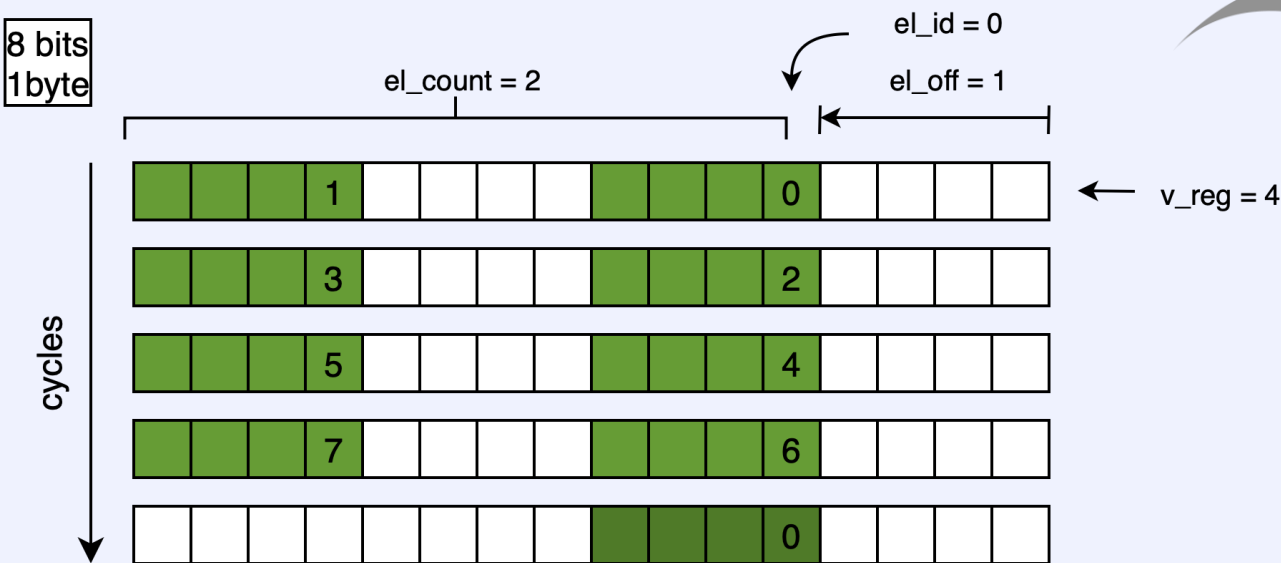
Writeback
Load Data

v0	<mask>								grp0
v1									grp1
v2									
v3									grp2
v4	7	6	5	4	3	2	1	0	
v5	t	t	5	4	3	2	1	0	grp3
v6									
v7									

VLEN = 256

Good Stride Packing Example

VPU will know the stride pattern sine it also decodes and stores the metadata



Larg/Mega Bobcat DMEM_W= 128bits

Writeback
Load Data

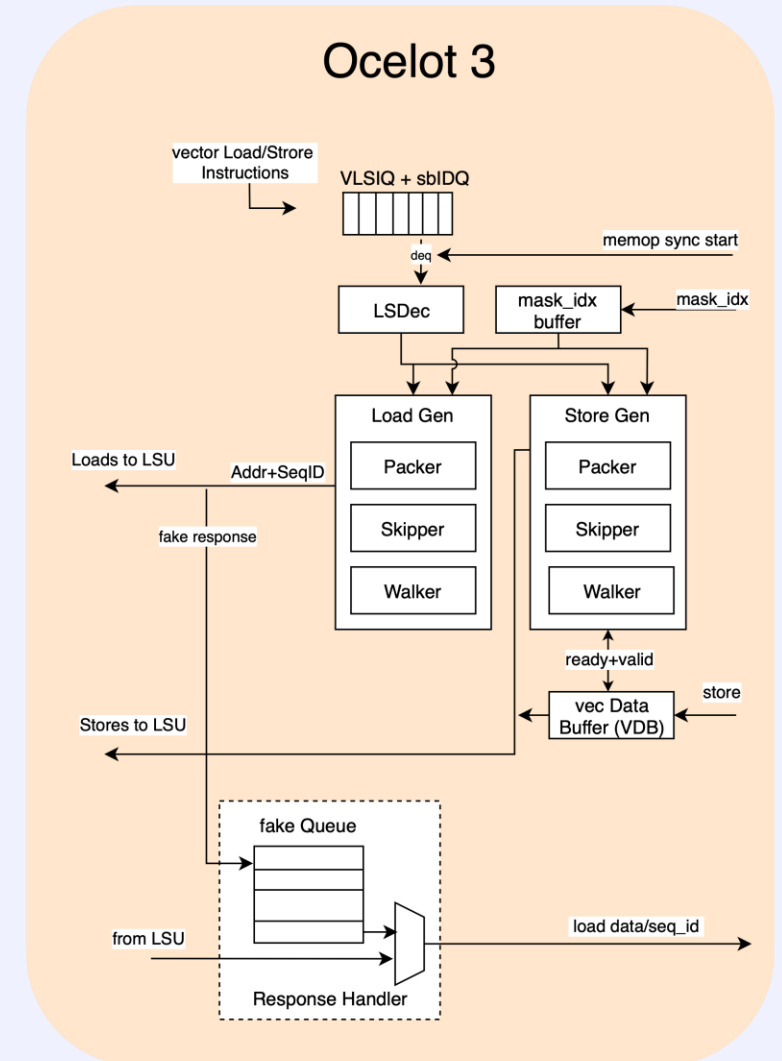
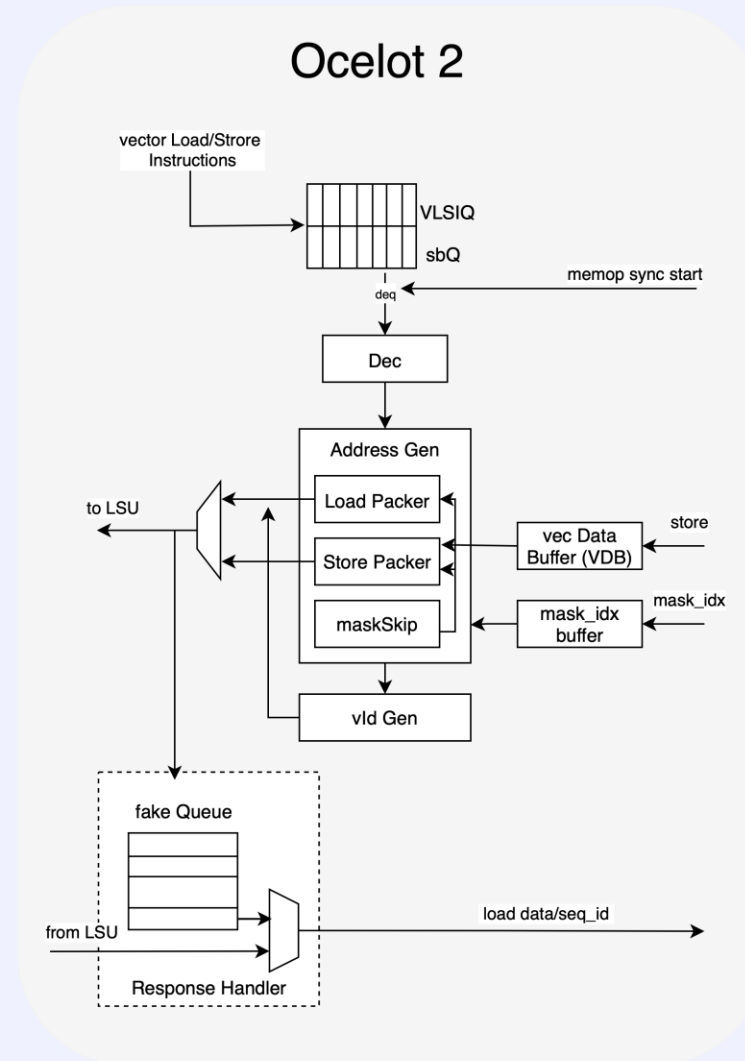
```
li x9, 9
vsetvli x8, x9, e32, m2, tu, mu
li x6, 0x80002014
li x7, 0x8
vlse32.v v4, (x6), x7
```

v0	<mask>								grp0
v1									
v2									grp1
v3									
v4	7	6	5	4	3	2	1	0	grp2
v5	t	t	t	t	t	t	t	0	
v6									grp3
v7									

VLEN = 256

Reconstruction of the Load-Store Address Generator

- Separate Load and Store Generators
- Better Organization of modules
- Helps with designing speculative load-stores
- Enabled Segment support
- Misalignment checking

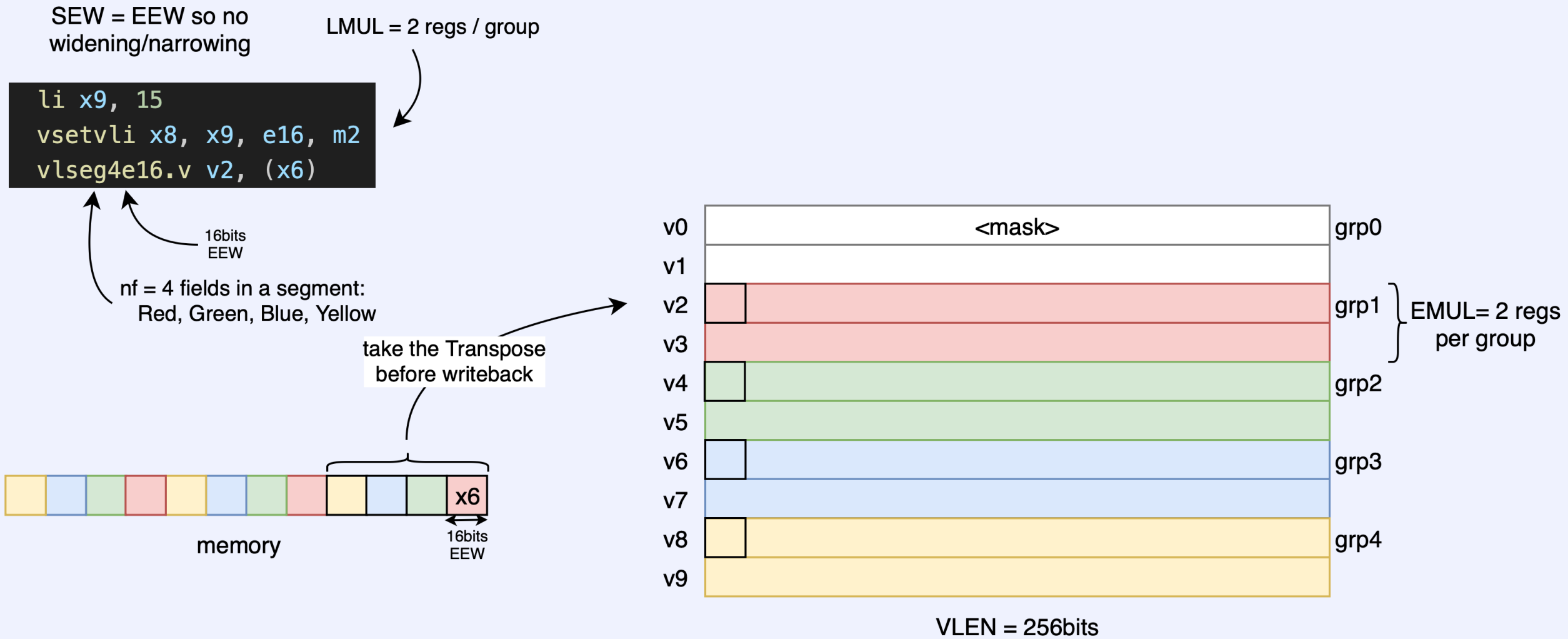




Support for Segment Load/Stores



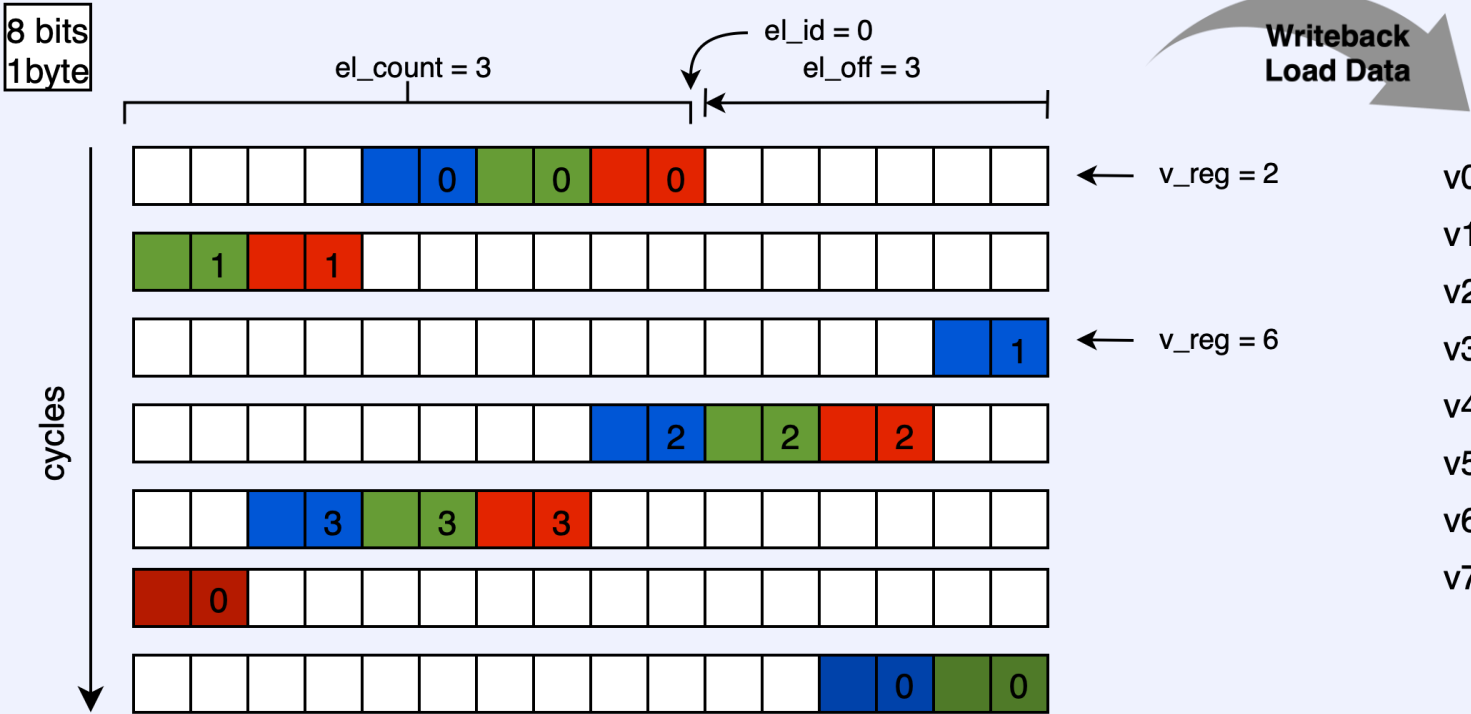
Segmented Load: The basics



Segmented Unit Stride Packing Example

we are writing to multiple registers from a packet for segmented, we can infer the destination with the starting v_reg
v_reg=6, EMUL=2, sb_vd=2 must mean that group_id=3, feild=2

```
li x9, 5
vsetvli x8, x9, e16, m2, tu, mu
li x6, 0x80002016
vlseg3e32.v v2, (x6)
```



v0	<mask>				grp0
v1					
v2	6	4	2	0	grp1
v3	t	t	t	0	
v4	6	4	2	0	grp2
v5	t	t	t	0	
v6	6	4	2	0	grp3
v7	t	t	t	0	

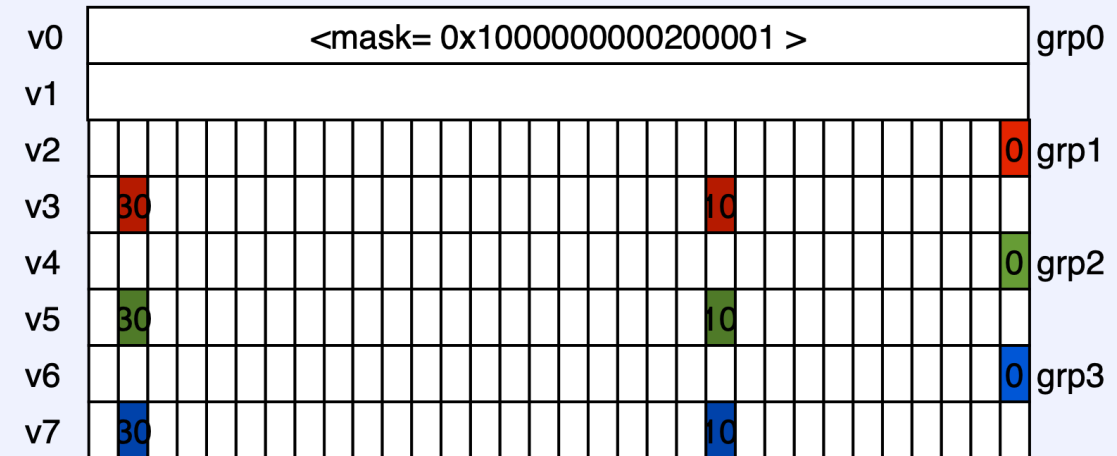
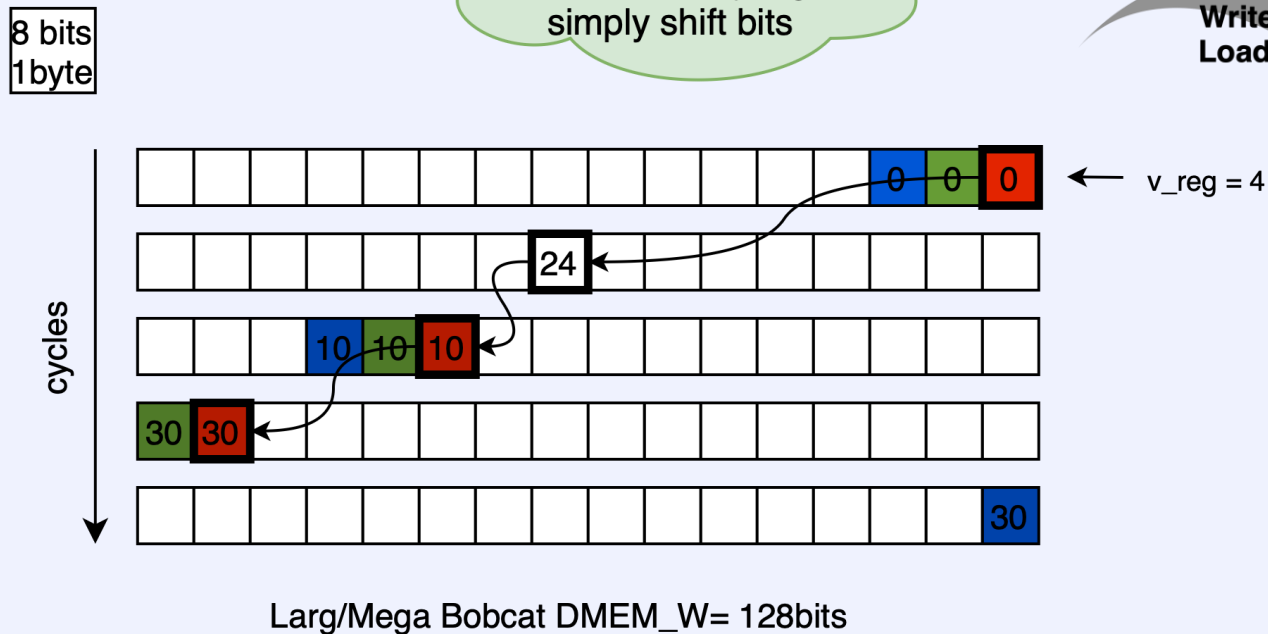
VLEN = 64 (example)

Segment Stride Skipping Example

Don't want to increment 64 bit address by using a multiplier. Force increment by log2 and simply shift bits

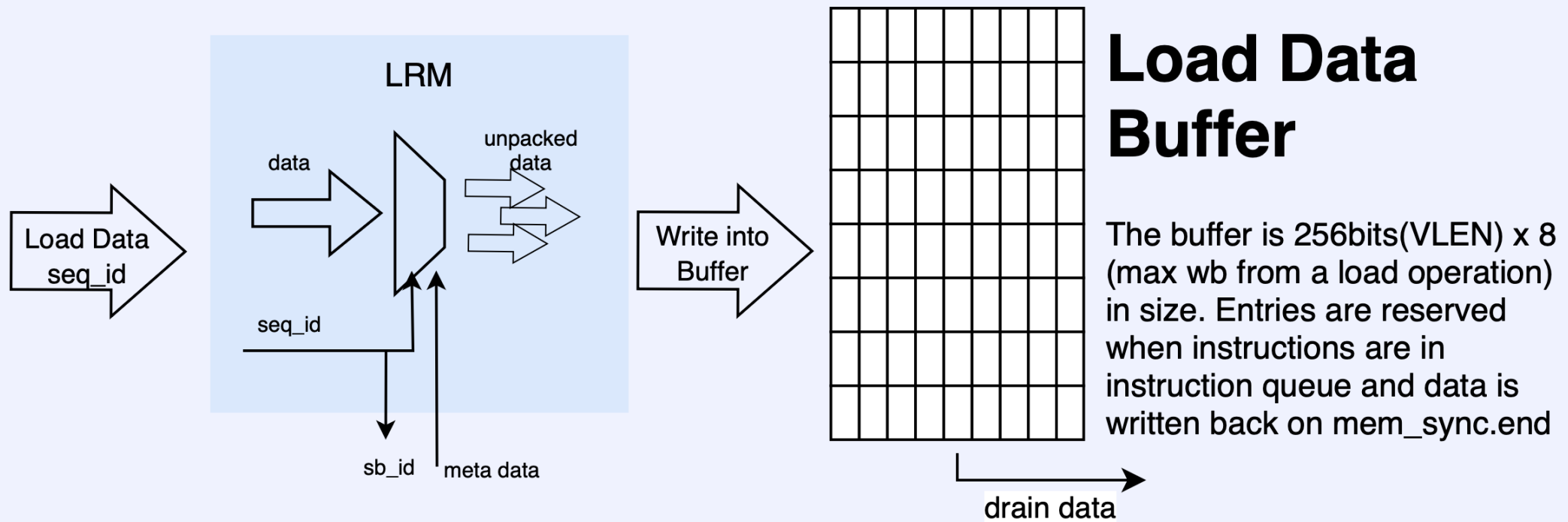
Writeback Load Data

```
li x9, 64
vsetvli x8, x9, e8, m2, tu, mu
li x6, 0x80002014
li x7, 0x32
vlsseg3e8.v v2, (x6), x7, v0.t
```



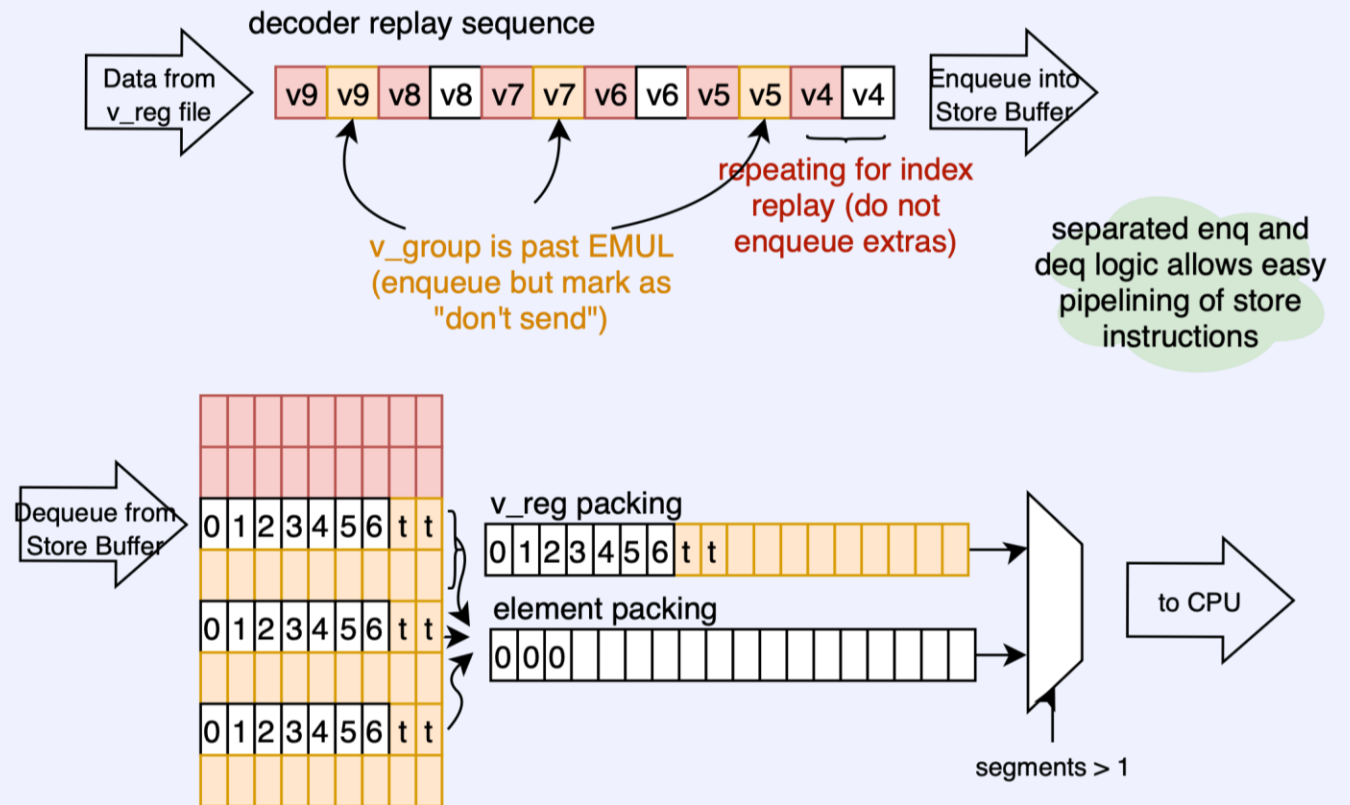
VLEN = 256

Load buffer in the VPU



Store Buffer in the VPU

```
// ==== Widening Index Store Example ====  
// Wide (2x)-index init  
li x9, 7 // vl = 7  
vsetvli x8, x9, e16, m4, tu, mu // LMUL = 4, SEW = 16  
vid.v v16 // v16 = [0,1,2,3..]  
// Store: 3 segments, 2 lmul (6 wb regs)  
vsetvli x8, x9, e8, m2, tu, mu // LMUL = 2, SEW = 8  
li x6, 0x80002010  
vsoxseg3ei32.v v4, (x6), v16
```





Future Roadmap: Precise Trap Support

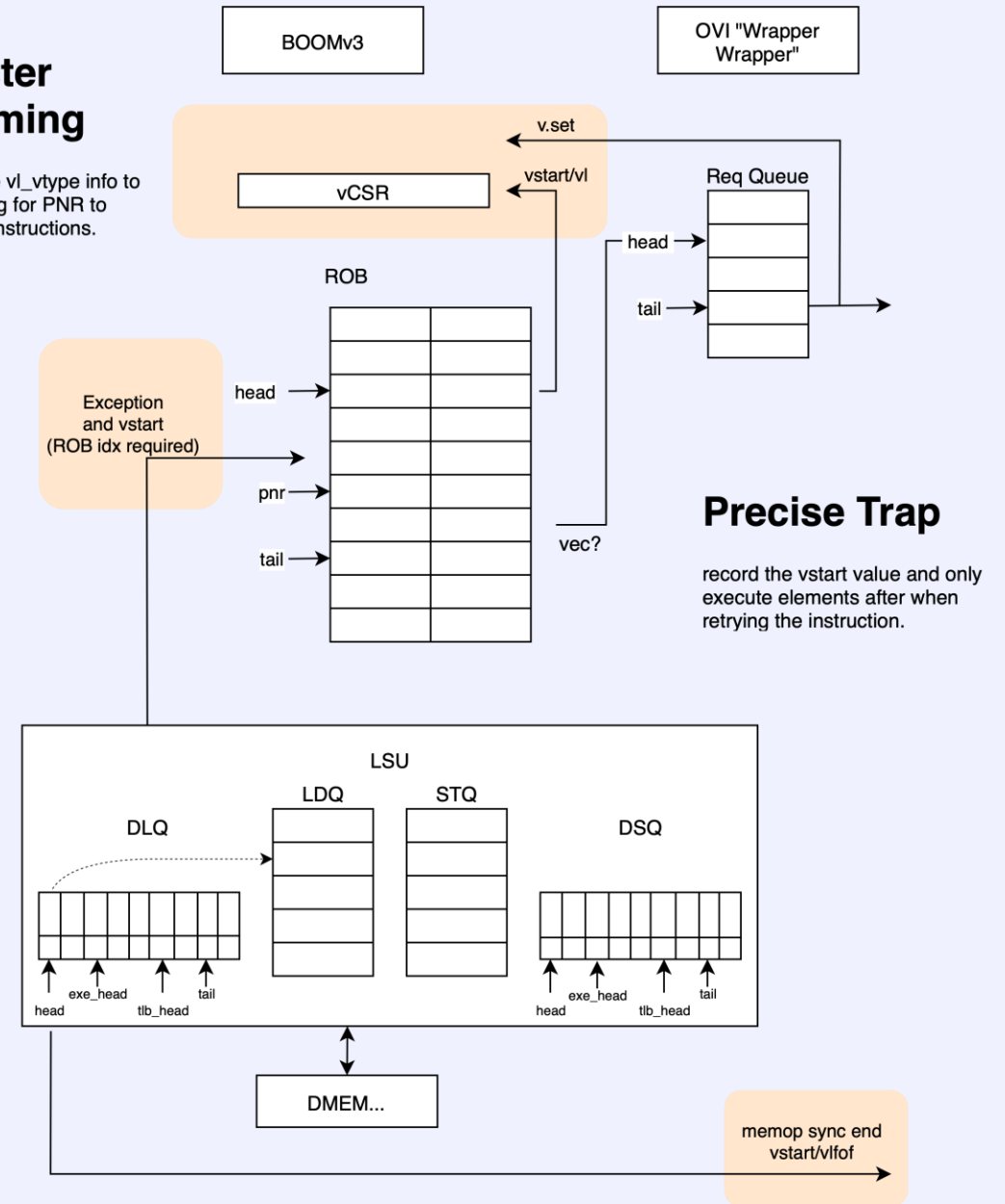


Future Plans

- Work on Precise Trap
- Add some renaming mechanism to the vector config instructions

Register Renaming

Rename the vl_vtype info to avoid waiting for PNR to reach vset instructions.





AI tools, Misc.



AI Tools

- Used Cursor for code explanation and scala syntax
- Used Cursor to generate test cases
- Used NotebookLM to tackle spec edge cases
- Trick: Fed output of NotebookLM as Cursor Prompt



Thank You

Special thanks:

- DJ
- Kuan-Yu Chen
- Alex White

